

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a602634a274abe7d4.jsonl`  
- SHA-256: `334a53b12a0df30929acb6b847e0caf8b3ec8ba027a5d30bc129178ecee99ccc`  
- Source modified: `2026-07-06T19:17:45+00:00`  
- Imported at: `2026-07-08T16:00:11+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:16:00.132Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy), src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

DIMENSION: accuracy of the ADR's concrete claims + internal consistency.

Investigate:

- SEED to verify/refute (I believe CONFIRMED): the "backwards-compatible: extra fields are ignored" claim is true
 - for READING (FilterConfig drops unknown keys ? verify with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe), BUT the SAME property means the existing

- save path silently ERASES the new keys: load_filter_config parses config_json into FilterConfig (dropping topics/allowlist/...), and save_filter_config re-serializes via FilterConfig.model_dump_json() ? so any /set
- command wipes the pipeline config. The ADR presents "ignored" purely as a positive and omits this data-loss
- interaction / the fact that F1 must extend BOTH parse and save paths. Verify against db.load_filter_config / save_filter_config + models.FilterConfig.
- The 'topics' example: prose says "stored as a JSON array" but the example is an OBJECT {include,exclude,mode};
 - and the example JSON contains a // comment (invalid JSON for real config_json). Confirm.
- Any other claim in the ADR that is false or internally inconsistent (tier costs, 'no migration', etc.).

Return findings.

2. assistant (2026-07-06T19:16:03.485Z)

I'll review the ADR and verify the concrete claims against the codebase.

3. user (2026-07-06T19:16:05.648Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept Current state
--- ---
`FilterConfig` / `Policy` One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.
`FilterEngine` Evaluates all rules; returns `FilterResult(passed, reasons)`.
`policies` table One row per account (is_default=1). `config_json` is the `FilterConfig` blob.
Security / topic / deep-inspect Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan.

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?                               Policy Pipeline                       ?
?                               ?                                     ?
? Tier 1 ? Metadata gates (always runs, zero I/O)                   ?
?   duration, resolution, size, codec, mime, GIF/round              ?
?   ? extends today's FilterEngine                                  ?
?                               ?                                     ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)                ?
?   source allowlist, keyword match, channel topic tags            ?
?   ? user-expressed interests filter the stream                   ?
?                               ?                                     ?
? Tier 3 ? Content safety (opt-in, metadata + light I/O)            ?
?   URL/domain reputation, file metadata sanity,                   ?
?   caption text scanning                                           ?
?   ? cheap checks; no download                                     ?
?                               ?                                     ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)                      ?
?   ffprobe, perceptual hash, downloaded content scan              ?
?   ? gated behind explicit user opt-in + budget                   ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...
```
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated

`StageResult` list is the **explanation trace** ? every decision is auditable.

Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config\_json` under a `topics` key:

```
```json
{
 "min_duration_s": 5,
 "topics": {
 "include": ["AI", "Philosophy", "Soccer"],
 "exclude": ["Politics", "Crypto"],
 "mode": "any" // "any" = match any include topic; "all" = match all
 }
}
```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted provider_source_ids).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content\_fingerprint` across accounts (already handled by `content\_items.dedup\_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked\_domain": "sketchy-site.com"}`).

Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep\_inspect: true` in `config\_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
 ctx: PolicyContext,
 stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
 results: list[StageResult] = []
 for stage in stages:
 result = await stage.evaluate(ctx)
 results.append(result)
```
```

```

        if not result.passed:
            return False, results # short-circuit
    return True, results
...

```

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

| Phase | Scope | Dependencies |
|---------------|-------------------------|--|
| **F1** | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler None |
| **F2** | Topic preferences | `topics` config parsing, source title/caption keyword matching F1 |
| **F3** | Source allowlist | `allowlist` config, skip non-allowlisted sources F1 |
| **F4** | URL/domain reputation | Domain blocklist, caption link extraction F1 |
| **F5** | Deep inspection | ffprobe/perceptual hash, download budget enforcement F1, ADR 0003 waiver |
| **F6** | Policy decisions table | Audit trail storage, explainability UI F1 |
| **F7** | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace F6 |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a

budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. user (2026-07-06T19:16:05.974Z)

```
1      """Pydantic data models for users, channels, videos, and filter config.
2
3      These models are the in-memory representation used by the repository layer
4      (`db.py`) and the filter engine (`filters/`). They are deliberately
5      decoupled from Telethon's own types so the filter engine stays pure and
6      unit-testable without a live Telegram connection.
7      """
8
9      from __future__ import annotations
10
11     from datetime import UTC, datetime
12     from enum import Enum
13
14     from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
15
16
17     def _utcnow() -> datetime:
18         """Timezone-aware UTC 'now', used as a default factory."""
```

```

19         return datetime.now(UTC)
20
21
22     class DeliveryMode(str, Enum):
23         """How a matched video is delivered to the user.
24
25         - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
26         - ``LINK``: send a text message with a t.me link + metadata summary.
27         - ``BOTH``: forward the video AND send the link message.
28         """
29
30         FORWARD = "forward"
31         LINK = "link"
32         BOTH = "both"
33
34
35     class Account(BaseModel):
36         """A product identity ? the owner of config, channels, routes, and content.
37
38         Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39         is the bridge used by the Bot API front-door (``from.id``) and by the
40         legacy userbot path; it is nullable so future providers that don't have a
41         Telegram id (e.g. WhatsApp) can still own an ``Account``.
42         """
43
44         model_config = ConfigDict(frozen=True)
45
46         id: int = Field(description="Stable surrogate PK; the owner key for all downstream
47         tables")
48         telegram_id: int | None = Field(
49             default=None,
50             description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
51             set.",
52         )
53         provider: str = Field(default="telegram", description="Provider namespace; future:
54         'whatsapp'")
55         display_name: str | None = None
56         created_at: datetime = Field(default_factory=_utcnow)
57
58     class TelegramSession(BaseModel):
59         """An optional MTPProto (Telethon) session linked to an :class:`Account`.
60
61         A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
62         that is what makes them structurally invisible to ``manager.run``
63         (ADR 0010). ``session_path`` is data-driven so the file-location
64         convention is explicit rather than derived from the PK at runtime.
65         """
66
67         model_config = ConfigDict(frozen=True)
68
69         account_id: int = Field(description="FK ? accounts.id; PK (one session per account
70         in v1)")
71         phone: str | None = Field(default=None, description="E.164 phone; used only for
72         bootstrap")
73         session_path: str = Field(description="File-backed Telethon session path
74         (user_<id>.session)")
75         enabled: bool = Field(
76             default=True,
77             description="If False, the manager skips this session (not the account)",
78         )
79         created_at: datetime = Field(default_factory=_utcnow)
80         last_login_at: datetime | None = None

```

```

78     class User(BaseModel):
79         """Deprecated: legacy v1 model conflating identity + MTPROTO session.
80
81         Retained as a backwards-compatible view over the new ``accounts`` +
82         ``telegram_sessions`` tables (ADR 0010). New code should use
83         :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84         maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85         and tests keep working without a flag day.
86         """
87
88         model_config = ConfigDict(frozen=True)
89
90         id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy
91 rows)")
92         phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
93         enabled: bool = Field(default=True, description="If False, the manager skips this
session")
94         created_at: datetime = Field(default_factory=_utcnow)
95
96     class Channel(BaseModel):
97         """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99         Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100 code should use :class:`Source` directly; the ``db`` layer maps
101 ``Channel`` ? ``Source`` so existing callers and tests keep working.
102         """
103
104         model_config = ConfigDict(frozen=True)
105
106         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
107         user_id: int
108         channel_id: int = Field(description="Telegram channel/chat id")
109         title: str | None = None
110         seen_first_at: datetime = Field(default_factory=_utcnow)
111
112     # --- provider-neutral domain models (ADR 0011, Track B) -----
113     #
114     # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
115     # shapes so the schema no longer leaks Telegram-specific ids into every future
116     # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
117     # make cross-provider collisions structurally impossible. See ADR 0011 for the
118     # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
119     # Option 2A: ``policies`` generalizes ``filter_config``).
120
121     class Source(BaseModel):
122         """A configured place we ingest content from (ADR 0011).
123
124         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
125         providers add their own source kinds). ``provider_source_id`` is the
126         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
127         """
128
129         model_config = ConfigDict(frozen=True)
130
131         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
132         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
133         provider: str = Field(default="telegram", description="Provider namespace")
134         provider_source_id: str = Field(
135             description="Provider-native source id (Telegram channel_id, ?)"
136         )
137         title: str | None = None

```



```

140     kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141     watched: bool = Field(
142         default=False,
143         description="ADR 0009 selective scanning: ingest from this source iff True",
144     )
145     seen_first_at: datetime = Field(default_factory=_utcnow)
146     last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
169 'article'")
170         text: str | None = Field(default=None, description="Caption / post text (nullable in
171 Phase 1)")
172         link_url: str | None = Field(default=None, description="For link/article posts
173 (Phase 1: NULL)")
174         media_ref: str | None = Field(default=None, description="Provider media reference")
175         document_id: str | None = Field(
176             default=None,
177             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
178         )
179         duration_s: int | None = None
180         width: int | None = None
181         height: int | None = None
182         size_bytes: int | None = None
183         mime_type: str | None = None
184         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
185 per account")
186         matched: bool = False
187         seen_at: datetime = Field(default_factory=_utcnow)
188
189     class Policy(BaseModel):
190         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
191
192         Generalizes the legacy per-account ``filter_config`` row so one account can
193         own multiple policies (enables ADR 0009's per-route filter overrides) and so
194         future enrichment/security stages grow into the same ``config_json`` column.
195         Exactly one policy per account carries ``is_default=True``; the legacy
196         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
197         """
198
199         model_config = ConfigDict(frozen=True)
200
201         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
202         account_id: int = Field(description="FK ? accounts.id")
203         name: str = Field(description="Policy name; 'default' for the account default

```

```

policy")
201     config_json: str = Field(
202         description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203     )
204     is_default: bool = Field(
205         default=False, description="Exactly one per account (UNIQUE partial index)"
206     )
207     created_at: datetime = Field(default_factory=_utcnow)
208     updated_at: datetime = Field(default_factory=_utcnow)
209
210
211 # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212 #
213 # These tables were created empty by Track B (_SCHEMA); Track C populates
214 # destinations (backfilled from filter_config_deprecated.config_json), writes
215 # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216 # defines the models for per-destination delivery state. The source/destination
217 # invariant (a channel is either source or destination, never both) is enforced
218 # at the app level by future selective-scanning logic (Track E, #30).
219
220
221 class Destination(BaseModel):
222     """A delivery target ? where matched content is sent (ADR 0011, Track C).
223
224     Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
225     can have multiple named output channels. Backfilled from the existing
226     per-account ``delivery_target``; the default destination is the target used
227     when no explicit routing rule matches.
228     """
229
230     model_config = ConfigDict(frozen=True)
231
232     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
233     account_id: int = Field(description="FK ? accounts.id")
234     provider: str = Field(default="telegram", description="Provider namespace")
235     target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
236     kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
237     title: str | None = None
238     is_output: bool = Field(
239         default=False,
240         description="ADR 0009: distinguishes output channels from ingest sources",
241     )
242     created_at: datetime = Field(default_factory=_utcnow)
243
244
245 class RoutingRule(BaseModel):
246     """A rule that maps a source through a policy to a destination (ADR 0009).
247
248     Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
249     ``policy_id`` is NULL, the account's default policy is used. When multiple
250     rules match, the highest ``priority`` wins.
251     """
252
253     model_config = ConfigDict(frozen=True)
254
255     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
256     account_id: int = Field(description="FK ? accounts.id")
257     source_id: int = Field(description="FK ? sources.id")
258     policy_id: int | None = Field(
259         default=None,
260         description="FK ? policies.id; NULL means use account default policy",
261     )
262     destination_id: int = Field(description="FK ? destinations.id")
263     enabled: bool = Field(default=True)
264     priority: int = Field(default=0, description="Higher wins when multiple rules

```

```

match")
265         created_at: datetime = Field(default_factory=_utcnow)
266
267
268     class DeliveryStatus(str, Enum):
269         """Per-destination delivery state, replacing the legacy binary ``forwarded``."""
270
271         PENDING = "pending"
272         DELIVERED = "delivered"
273         FAILED = "failed"
274         RETRYING = "retrying"
275
276
277     class DeliveryAttempt(BaseModel):
278         """Per-destination delivery record ? idempotent via UNIQUE index.
279
280         Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
281         ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
282         incremented and ``status``/``last_error`` are updated.
283         """
284
285         model_config = ConfigDict(frozen=True)
286
287         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
288         content_item_id: int = Field(description="FK ? content_items.id")
289         destination_id: int = Field(description="FK ? destinations.id")
290         route_id: int | None = Field(default=None, description="FK ? routing_rules.id
(nullable)")
291         status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
292         attempt_count: int = Field(default=0, description="Incremented on each retry")
293         last_error: str | None = None
294         created_at: datetime = Field(default_factory=_utcnow)
295         delivered_at: datetime | None = None
296         next_retry_at: datetime | None = None
297
298
299     class FilterConfig(BaseModel):
300         """Per-user filter configuration.
301
302         v1 rules: duration bounds, minimum height (resolution), and max file size.
303         Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
304         new field later is backwards-compatible.
305         """
306
307         # Duration bounds in seconds. None means unbounded on that side.
308         min_duration_s: int | None = Field(default=5, ge=0)
309         max_duration_s: int | None = Field(default=7200, ge=0)
310
311         # Resolution: a video passes if its height >= target_height.
312         # None disables the resolution rule.
313         target_height: int | None = Field(default=1080, ge=1)
314
315         # Max file size in mebibytes. None disables the size rule.
316         max_size_mb: int | None = Field(default=2048, ge=1)
317
318         # Content-type exclusions. GIFs and round videos are excluded by default
319         # (their duration/dimensions are usually incidental to real video filters);
320         # silent videos are off by default since some legit clips have no audio.
321         # Existing users' stored configs are unaffected ? the DB JSON blob overrides
322         # these defaults, so this only changes the behaviour for fresh installs.
323         exclude_gif: bool = Field(
324             default=True,
325             description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
326         )
327         exclude_round: bool = Field(

```

```

328         default=True,
329         description="If True, exclude circular video messages (round_message=True)",
330     )
331     exclude_nosound: bool = Field(
332         default=False,
333         description="If True, exclude videos without an audio track (nosound=True)",
334     )
335
336     # How matched videos are delivered: forward the video, send a link, or both.
337     # Stored in the same JSON blob, so adding this field is backwards-compatible
338     # (existing rows without it default to FORWARD).
339     delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
340
341     # Where matched videos / link messages are sent. Accepts:
342     # - "me" (default): Saved Messages ? exact v1.1 behaviour.
343     # - "@username": a public channel the account is a member of.
344     # - "-100..." or a bare int-as-string: a numeric Telegram chat id
345     #   (private channels without a username).
346     # The string is passed verbatim to Telethon's forward_messages / send_message.
347     # See ADR 0005 for rationale and access requirements.
348     delivery_target: str = Field(
349         default="me",
350         min_length=1,
351         description="Telethon entity to deliver to: 'me', '@username', or a chat id",
352     )
353
354     @field_validator("delivery_target", mode="before")
355     @classmethod
356     def _coerce_delivery_target(cls, v: object) -> str:
357         """Accept int chat ids (e.g. -1001234567890) and coerce to str.
358
359         Telethon accepts both str and int as an entity, but storing as str keeps
360         the JSON blob uniform and lets '@username' / 'me' / numeric ids share
361         one field type.
362         """
363         if v is None:
364             return "me"
365         if isinstance(v, int):
366             return str(v)
367         return str(v)
368
369     @model_validator(mode="after")
370     def _check_duration_bounds(self) -> FilterConfig:
371         """Ensure min_duration_s <= max_duration_s when both are set."""
372         if (
373             self.min_duration_s is not None
374             and self.max_duration_s is not None
375             and self.min_duration_s > self.max_duration_s
376         ):
377             raise ValueError(
378                 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
379                 f"({self.max_duration_s})"
380             )
381         return self
382
383     class VideoRecord(BaseModel):
384         """A seen video message + its metadata. Drives dedup and future /search.
385
386         Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
387         New code should use :class:`ContentItem`; the ``db`` layer maps
388         ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
389         """
390
391         model_config = ConfigDict(frozen=True)

```

```

392
393     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
394     user_id: int
395     channel_id: int = Field(description="Origin Telegram channel id")
396     message_id: int = Field(description="Telegram message id within the channel")
397     document_id: int | None = Field(
398         default=None,
399         description="Telegram document.id ? reused across forwards; Tier-1 content
dedup",
400     )
401
402     # Metadata extracted from the Telethon message (may be None if absent).
403     duration_s: int | None = None
404     width: int | None = None
405     height: int | None = None
406     size_bytes: int | None = None
407     mime_type: str | None = None
408
409     matched: bool = False
410     forwarded: bool = False
411     seen_at: datetime = Field(default_factory=_utcnow)
412
413     @property
414     def dedup_key(self) -> tuple[int, int, int]:
415         """The natural key used for deduplication in the ``videos`` table."""
416         return (self.user_id, self.channel_id, self.message_id)
417
418     @property
419     def content_fingerprint(self) -> tuple[int, int, int, int] | None:
420         """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height,
width).
421
422         Returns None if any field is missing (cannot fingerprint).
423         """
424         if None in (self.size_bytes, self.duration_s, self.height, self.width):
425             return None
426         return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]
427
428
429     # Default filter config shared by the bootstrap onboarding step.
430     DEFAULT_FILTER_CONFIG = FilterConfig()
431

```

5. user (2026-07-06T19:16:06.497Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1389 of 1817 total (27782 tokens, cap 25000). Call Read with offset=1390 limit=1389 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1     """SQLite persistence layer (aiosqlite).
2
3     Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
4     the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``
5     for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
6     ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8     The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9     in-memory database (``aiosqlite.connect(":memory:")``).
10    """
11
12    from __future__ import annotations
13
14    from collections.abc import Awaitable, Callable, Iterable

```

```

15     from datetime import UTC, datetime
16
17     import aiosqlite
18
19     from tg_video_filter.models import (
20         Account,
21         Channel,
22         ContentItem,
23         DeliveryAttempt,
24         DeliveryStatus,
25         Destination,
26         FilterConfig,
27         Policy,
28         RoutingRule,
29         Source,
30         TelegramSession,
31         User,
32         VideoRecord,
33     )
34
35     # --- DDL -----
36
37     _SCHEMA = """
38     CREATE TABLE IF NOT EXISTS accounts (
39         id            INTEGER PRIMARY KEY,
40         telegram_id   INTEGER UNIQUE,
41         provider      TEXT NOT NULL DEFAULT 'telegram',
42         display_name  TEXT,
43         created_at    TEXT NOT NULL
44     );
45
46     CREATE TABLE IF NOT EXISTS telegram_sessions (
47         account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
48         phone         TEXT,
49         session_path  TEXT NOT NULL,
50         enabled       INTEGER NOT NULL DEFAULT 1,
51         created_at    TEXT NOT NULL,
52         last_login_at TEXT
53     );
54
55     CREATE TABLE IF NOT EXISTS channels (
56         id            INTEGER PRIMARY KEY AUTOINCREMENT,
57         user_id       INTEGER NOT NULL REFERENCES accounts(id),
58         channel_id    INTEGER NOT NULL,
59         title         TEXT,
60         seen_first_at TEXT NOT NULL,
61         UNIQUE (user_id, channel_id)
62     );
63
64     CREATE TABLE IF NOT EXISTS videos (
65         id            INTEGER PRIMARY KEY AUTOINCREMENT,
66         user_id       INTEGER NOT NULL REFERENCES accounts(id),
67         channel_id    INTEGER NOT NULL,
68         message_id    INTEGER NOT NULL,
69         document_id   INTEGER,
70         duration_s    INTEGER,
71         width         INTEGER,
72         height        INTEGER,
73         size_bytes    INTEGER,
74         mime_type     TEXT,
75         matched       INTEGER NOT NULL DEFAULT 0,
76         forwarded     INTEGER NOT NULL DEFAULT 0,
77         seen_at       TEXT NOT NULL,
78         UNIQUE (user_id, channel_id, message_id)
79     );

```

```

80
81 CREATE TABLE IF NOT EXISTS filter_config (
82     user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
83     config_json  TEXT NOT NULL
84 );
85
86 -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
87 -- ``content_items``, and ``policies`` are populated by migration 003 (and by
88 -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
89 -- are created empty here so the schema is complete ? Track C (#28) implements
90 -- the delivery logic that populates them.
91
92 CREATE TABLE IF NOT EXISTS sources (
93     id            INTEGER PRIMARY KEY AUTOINCREMENT,
94     account_id    INTEGER NOT NULL REFERENCES accounts(id),
95     provider      TEXT NOT NULL DEFAULT 'telegram',
96     provider_source_id TEXT NOT NULL,
97     title         TEXT,
98     kind          TEXT NOT NULL DEFAULT 'channel',
99     watched       INTEGER NOT NULL DEFAULT 0,
100    seen_first_at  TEXT NOT NULL,
101    last_seen_at   TEXT,
102    UNIQUE (account_id, provider, provider_source_id)
103 );
104
105 CREATE TABLE IF NOT EXISTS content_items (
106     id            INTEGER PRIMARY KEY AUTOINCREMENT,
107     account_id    INTEGER NOT NULL REFERENCES accounts(id),
108     source_id     INTEGER NOT NULL REFERENCES sources(id),
109     provider      TEXT NOT NULL DEFAULT 'telegram',
110     provider_message_id TEXT NOT NULL,
111     content_type  TEXT NOT NULL DEFAULT 'video',
112     text          TEXT,
113     link_url      TEXT,
114     media_ref     TEXT,
115     document_id   TEXT,
116     duration_s    INTEGER,
117     width         INTEGER,
118     height        INTEGER,
119     size_bytes    INTEGER,
120     mime_type     TEXT,
121     dedup_key     TEXT NOT NULL,
122     matched       INTEGER NOT NULL DEFAULT 0,
123     seen_at       TEXT NOT NULL,
124     UNIQUE (account_id, source_id, provider_message_id),
125     UNIQUE (account_id, dedup_key)
126 );
127
128 CREATE TABLE IF NOT EXISTS policies (
129     id            INTEGER PRIMARY KEY AUTOINCREMENT,
130     account_id    INTEGER NOT NULL REFERENCES accounts(id),
131     name          TEXT NOT NULL,
132     config_json  TEXT NOT NULL,
133     is_default    INTEGER NOT NULL DEFAULT 0,
134     created_at   TEXT NOT NULL,
135     updated_at   TEXT NOT NULL,
136     UNIQUE (account_id, name)
137 );
138
139 -- ADR 0011: exactly one default policy per account (structural invariant).
140 CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
141 ON policies(account_id) WHERE is_default = 1;
142
143 CREATE TABLE IF NOT EXISTS destinations (
144     id            INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

145     account_id  INTEGER NOT NULL REFERENCES accounts(id),
146     provider    TEXT NOT NULL DEFAULT 'telegram',
147     target      TEXT NOT NULL,
148     kind        TEXT NOT NULL DEFAULT 'saved',
149     title       TEXT,
150     is_output    INTEGER NOT NULL DEFAULT 0,
151     created_at   TEXT NOT NULL,
152     UNIQUE (account_id, provider, target)
153 );
154
155 CREATE TABLE IF NOT EXISTS routing_rules (
156     id           INTEGER PRIMARY KEY AUTOINCREMENT,
157     account_id   INTEGER NOT NULL REFERENCES accounts(id),
158     source_id    INTEGER NOT NULL REFERENCES sources(id),
159     policy_id    INTEGER REFERENCES policies(id),
160     destination_id INTEGER NOT NULL REFERENCES destinations(id),
161     enabled      INTEGER NOT NULL DEFAULT 1,
162     priority     INTEGER NOT NULL DEFAULT 0,
163     created_at   TEXT NOT NULL,
164     UNIQUE (account_id, source_id, destination_id)
165 );
166
167 CREATE TABLE IF NOT EXISTS delivery_attempts (
168     id           INTEGER PRIMARY KEY AUTOINCREMENT,
169     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
170     destination_id INTEGER NOT NULL REFERENCES destinations(id),
171     route_id     INTEGER REFERENCES routing_rules(id),
172     status       TEXT NOT NULL DEFAULT 'pending',
173     attempt_count INTEGER NOT NULL DEFAULT 0,
174     last_error   TEXT,
175     created_at   TEXT NOT NULL,
176     delivered_at TEXT,
177     next_retry_at TEXT,
178     UNIQUE (content_item_id, destination_id)
179 );
180 ""
181
182 # Note: the ``idx_videos_user_document`` partial unique index is created by
183 # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
184 # which is present on fresh tables but only added to legacy v1 DBs by the
185 # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
186 # DBs that don't yet have the column.
187
188
189 async def init_db(db_path: str) -> None:
190     """Open the database at ``db_path`` and create/migrate the schema.
191
192     Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
193     then runs any pending migrations via the ``user_version`` ladder.
194     """
195     async with aiosqlite.connect(db_path) as conn:
196         await _apply_schema(conn)
197
198
199 async def _apply_schema(conn: aiosqlite.Connection) -> None:
200     # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
201     # and a busy timeout so concurrent access waits instead of failing immediately.
202     await conn.execute("PRAGMA journal_mode = WAL")
203     await conn.execute("PRAGMA foreign_keys = ON")
204     await conn.execute("PRAGMA busy_timeout = 5000")
205     await conn.executescript(_SCHEMA)
206     # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
207     # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
208     # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
209     # Once migration 003 has run (signalled by ``channels_deprecated`` existing),

```



```

210     # any *empty* recreations of those legacy tables from a subsequent
211     # ``init_db`` are dropped here so they don't shadow the renamed copies.
212     if await _table_exists(conn, "channels_deprecated"):
213         await conn.execute("DROP TABLE IF EXISTS channels")
214         await conn.execute("DROP TABLE IF EXISTS videos")
215         await conn.execute("DROP TABLE IF EXISTS filter_config")
216     await _apply_migrations(conn)
217     await conn.commit()
218
219
220     # --- migrations -----
221     #
222     # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
223     # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
224     # the source of truth for *fresh* databases; the migrations below reconcile
225     # *existing* databases that were created by an older schema version.
226     #
227     # To add a new migration:
228     #   1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
229     #   2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
230     #   3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
231     #       ``init_db``, and asserts the new structure.
232
233     _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
234
235
236     def _migration(
237         version: int,
238     ) -> Callable[
239         [Callable[[aiosqlite.Connection], Awaitable[None]]],
240         Callable[[aiosqlite.Connection], Awaitable[None]],
241     ]:
242         """Register a migration that brings the DB up to ``user_version == version``."""
243
244         def decorator(
245             fn: Callable[[aiosqlite.Connection], Awaitable[None]],
246         ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
247             _MIGRATIONS.append((version, fn))
248             return fn
249
250         return decorator
251
252
253     # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
254     #
255     # Pure function: computes the ``content_items.dedup_key`` value from metadata
256     # already extracted at ingestion time (no download ? honors ADR 0003). The
257     # decision tree is ordered strongest-key first; each branch is guarded so
258     # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0:0``).
259     # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
260
261
262     def compute_dedup_key(
263         *,
264         provider: str,
265         document_id: str | None = None,
266         source_id: int | str | None = None,
267         provider_message_id: str | None = None,
268         size_bytes: int | None = None,
269         duration_s: int | None = None,
270         height: int | None = None,
271         width: int | None = None,
272     ) -> str:
273         """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
274

```

```

275 Implements the ADR 0011 Option 1A decision tree:
276
277 - **Telegram** (`provider == "telegram"`):
278     1. If `document_id` is set ? `tg:doc:{document_id}`.
279     2. Else if **all four** fingerprint fields (size, duration, height, width)
280        are set ? `tg:fp:{size}:{dur}:{h}:{w}`.
281     3. Else ? `tg:msg:{source_id}:{provider_message_id}` (Tier-0 fallback;
282        `source_id` is required because Telegram `message_id` is only
283        unique within a channel).
284 - **Other providers** (`provider != "telegram"`): fall back to the Tier-0
285    `{provider}:msg:{source_id}:{provider_message_id}` shape. Future
286    providers add their own branches (e.g. `wa:sha256:{blob_hash}`) here.
287
288 Raises `ValueError` if the fallback branch is reached without the fields
289 it requires (`source_id` + `provider_message_id`) ? a degenerate
290 `dedup_key` is worse than a loud failure at ingestion time.
291 """
292 p = provider.lower()
293 if p == "telegram":
294     if document_id is not None and document_id != "":
295         return f"tg:doc:{document_id}"
296     # All four fingerprint fields are required to avoid a degenerate
297     # `tg:fp:0:0:0:0` key when some metadata is missing.
298     if None not in (size_bytes, duration_s, height, width):
299         return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
300     # Tier-0 fallback.
301     if source_id is None or provider_message_id is None:
302         raise ValueError(
303             "Telegram dedup_key fallback requires source_id and provider_message_id"
304         )
305     return f"tg:msg:{source_id}:{provider_message_id}"
306 # Future-proof default for providers without a dedicated branch.
307 if source_id is None or provider_message_id is None:
308     raise ValueError(
309         f"dedup_key for provider={provider!r} requires source_id and
310 provider_message_id"
311     )
312 return f"{p}:msg:{source_id}:{provider_message_id}"
313
314
315 async def _apply_migrations(conn: aiosqlite.Connection) -> None:
316     """Run every registered migration whose version > current `user_version`."""
317     async with conn.execute("PRAGMA user_version") as cur:
318         row = await cur.fetchone()
319         current = int(row[0]) if row else 0
320
321     for version, fn in _MIGRATIONS:
322         if version <= current:
323             continue
324         await fn(conn)
325         await conn.execute(f"PRAGMA user_version = {version}")
326
327
328 async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
329     """Return True if `column` exists on `table` (async, PRAGMA table_info)."""
330     async with conn.execute(f"PRAGMA table_info({table})") as cur:
331         rows = await cur.fetchall()
332         return any(row[1] == column for row in rows)
333
334
335 @migration(1)
336 async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
337     """Add `document_id` column + partial unique index (ADR 0006 content dedup).

```

```

338
339 Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
340 created before this migration existed.
341 """
342 if not await _column_exists(conn, "videos", "document_id"):
343     await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
344 # Create the partial unique index for both fresh and migrated DBs. Uses
345 # IF NOT EXISTS so it is safe on DBs where it was already created.
346 await conn.execute(
347     "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
348     "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
349 )
350
351
352 @_migration(2)
353 async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
354     """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
355
356     ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
357     data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
358     so their integer ``user_id`` values still resolve to the same identity (now
359     via ``accounts.id``). The legacy ``users`` table is renamed to
360     ``users_deprecated`` and kept for one release as a rollback safety net; a
361     follow-up migration drops it.
362
363     FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
364     ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape
365     as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting
366     those FKs to point at ``users_deprecated``, diverging from fresh installs
367     and breaking inserts for new accounts ? see PR #33 review.)
368
369     Idempotent: a no-op once the legacy ``users`` table has been renamed.
370     """
371     # If the legacy users table is gone, this migration has already run
372     # (or the DB is fresh and only has accounts). Nothing to do.
373     if not await _table_exists(conn, "users"):
374         return
375
376     # Build accounts from the legacy users table. accounts.id := users.id.
377     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
378     # and recreated here so the INSERT can populate it cleanly.)
379     await conn.execute("DROP TABLE IF EXISTS accounts")
380     await conn.execute(
381         """
382         CREATE TABLE accounts (
383             id            INTEGER PRIMARY KEY,
384             telegram_id   INTEGER UNIQUE,
385             provider      TEXT NOT NULL DEFAULT 'telegram',
386             display_name  TEXT,
387             created_at    TEXT NOT NULL
388         )
389         """
390     )
391     await conn.execute(
392         """
393         INSERT INTO accounts (id, telegram_id, provider, created_at)
394         SELECT id, id, 'telegram', created_at FROM users
395         """
396     )
397
398     # Build telegram_sessions, one row per legacy userbot user.
399     # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
400     await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
401     await conn.execute(
402         """

```

```

403         CREATE TABLE telegram_sessions (
404             account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
405             phone          TEXT,
406             session_path  TEXT NOT NULL,
407             enabled        INTEGER NOT NULL DEFAULT 1,
408             created_at     TEXT NOT NULL,
409             last_login_at  TEXT
410         )
411         """"
412     )
413     # session_path is built from the *telegram_id* (= old users.id) to match
414     # the convention telethon_client.build_client used pre-migration. The
415     # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
416     # so this migration does not need to know the runtime path.
417     await conn.execute(
418         """"
419         INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
created_at)
420         SELECT id, phone, 'user_' || id || '.session', enabled, created_at
421         FROM users
422         """"
423     )
424
425     # Rebuild channels / videos / filter_config so their FK constraints point
426     # at accounts(id) instead of the legacy users(id). SQLite's
427     # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
428     # the rename ? so a naive rename of users ? users_deprecated would leave
429     # these tables referencing users_deprecated(id), diverging from fresh DBs
430     # (which reference accounts(id)) and breaking inserts for new accounts.
431     # The rebuild: rename the data-holding legacy table out of the way, create
432     # the new one with the correct FK, copy the data back, drop the renamed copy.
433     for table, new_ddl, copy_cols in (
434         (
435             "channels",
436             """"
437             CREATE TABLE channels (
438                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
439                 user_id        INTEGER NOT NULL REFERENCES accounts(id),
440                 channel_id     INTEGER NOT NULL,
441                 title          TEXT,
442                 seen_first_at  TEXT NOT NULL,
443                 UNIQUE (user_id, channel_id)
444             )
445             """,
446             "id, user_id, channel_id, title, seen_first_at",
447         ),
448         (
449             "videos",
450             """"
451             CREATE TABLE videos (
452                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
453                 user_id        INTEGER NOT NULL REFERENCES accounts(id),
454                 channel_id     INTEGER NOT NULL,
455                 message_id     INTEGER NOT NULL,
456                 document_id    INTEGER,
457                 duration_s     INTEGER,
458                 width          INTEGER,
459                 height         INTEGER,
460                 size_bytes     INTEGER,
461                 mime_type      TEXT,
462                 matched        INTEGER NOT NULL DEFAULT 0,
463                 forwarded      INTEGER NOT NULL DEFAULT 0,
464                 seen_at        TEXT NOT NULL,
465                 UNIQUE (user_id, channel_id, message_id)
466             )

```

```

467         """
468         "id, user_id, channel_id, message_id, document_id, duration_s, width, "
469         "height, size_bytes, mime_type, matched, forwarded, seen_at",
470     ),
471     (
472         "filter_config",
473         """
474         CREATE TABLE filter_config (
475             user_id      INTEGER PRIMARY KEY REFERENCES accounts(id),
476             config_json  TEXT NOT NULL
477         )
478         """
479         "user_id, config_json",
480     ),
481 ):
482     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
483     await conn.execute(new_ddl)
484     await conn.execute(
485         f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
486     )
487     await conn.execute(f"DROP TABLE {table}__legacy_data")
488
489     # The partial unique index for document_id dedup (created by migration 001
490     # against the legacy videos table) was dropped along with the legacy copy
491     # above. Recreate it on the new videos table.
492     await conn.execute(
493         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
494         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
495     )
496
497     # Rename the legacy users table (do NOT drop ? kept as a rollback safety
498     # net for one release per ADR 0010). The data-holding channels/videos/
499     # filter_config tables were already rebuilt above with FKs to accounts(id),
500     # so renaming users no longer retargets any live FK.
501     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
502
503
504 @_migration(3)
505 async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
506     """Land the provider-neutral domain tables (ADR 0011, Track B).
507
508     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
509     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
510     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
511     them), then backfills the three populated ones from the legacy
512     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
513     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
514     pattern). Idempotent: a no-op once the legacy tables have been renamed.
515     """
516     # Idempotency gate: if the legacy videos table is gone, this migration has
517     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
518     # dropped by _apply_schema post-003). Nothing to backfill.
519     if not await _table_exists(conn, "videos"):
520         return
521
522     # 1a. Backfill sources from channels first (known title + watched=1). Order
523     # matters: channels-first so title/watched data wins; the videos-based
524     # backfill below fills gaps for orphan channels (review finding #1).
525     await conn.execute(
526         """
527         INSERT OR IGNORE INTO sources
528             (account_id, provider, provider_source_id, title, kind, watched,
529              seen_first_at, last_seen_at)
530         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',

```

```

531         1, seen_first_at, seen_first_at
532     FROM channels
533     """"
534 )
535
536 # 1b. Backfill sources from videos for channels that have no channels row
537 #     (e.g. backfill-only channels that were never seen via live traffic).
538 #     INSERT OR IGNORE is a no-op on (account, provider, source_id) conflicts
539 #     ? the channels-based row above survives with its title + watched=1.
540 await conn.execute(
541     """"
542     INSERT OR IGNORE INTO sources
543         (account_id, provider, provider_source_id, kind, watched, seen_first_at)
544     SELECT v.user_id, 'telegram', CAST(v.channel_id AS TEXT),
545            'channel', 0, MIN(v.seen_at)
546     FROM videos v
547     GROUP BY v.user_id, v.channel_id
548     """"
549 )
550
551 # 2. Backfill content_items from videos in two phases (review findings #2 & #3):
552 #     Phase A ? insert every row with a guaranteed-unique tg:msg: Tier-0 key
553 #     (no silent drops, even on fingerprint collisions).
554 #     Phase B ? promote to tg:doc: for items with a document_id (one per doc-id
555 group).
556 #     Phase C ? promote remaining non-doc items to tg:fp: (one per fingerprint
557 group).
558 #     The non-promoted losers stay at tg:msg: (unique) ? no silent data loss.
559 await conn.execute(
560     """"
561     INSERT INTO content_items
562         (account_id, source_id, provider, provider_message_id, content_type,
563          document_id, duration_s, width, height, size_bytes, mime_type,
564          dedup_key, matched, seen_at)
565     SELECT
566         v.user_id,
567         s.id,
568         'telegram',
569         CAST(v.message_id AS TEXT),
570         'video',
571         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
572         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
573         'tg:msg:' || s.id || ':' || v.message_id, -- Tier-0 (always unique)
574         v.matched,
575         v.seen_at
576     FROM videos v
577     JOIN sources s
578         ON s.account_id = v.user_id
579         AND s.provider = 'telegram'
580         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
581     """"
582 )
583
584 # Phase B: promote one row per (account, document_id) to tg:doc:.
585 await conn.execute(
586     """"
587     UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
588     WHERE document_id IS NOT NULL
589     AND dedup_key LIKE 'tg:msg:%'
590     AND id IN (
591         SELECT MIN(id) FROM content_items
592         WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
593         GROUP BY account_id, document_id
594     )
595     """"

```

```

594 )
595
596 # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
597 await conn.execute(
598     """
599     UPDATE content_items SET dedup_key =
600         'tg:fp:' || size_bytes || ':' || duration_s || ':'
601         || height || ':' || width
602     WHERE document_id IS NULL
603         AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
604         AND height IS NOT NULL AND width IS NOT NULL
605         AND dedup_key LIKE 'tg:msg:%'
606         AND id IN (
607             SELECT MIN(id) FROM content_items
608             WHERE document_id IS NULL
609                 AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
610                 AND height IS NOT NULL AND width IS NOT NULL
611                 AND dedup_key LIKE 'tg:msg:%'
612             GROUP BY account_id, size_bytes, duration_s, height, width
613         )
614     """
615 )
616
617 # 3. Backfill policies from filter_config: one 'default' policy per account
618 # (Option 2A). is_default=1 so the legacy load_filter_config shim reads
619 # this row. UNIQUE(account_id, name) dedups idempotent re-runs.
620 await conn.execute(
621     """
622     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
623                                     created_at, updated_at)
624     SELECT user_id, 'default', config_json, 1,
625            strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
626            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
627     FROM filter_config
628     """
629 )
630
631 # 4. Create the policies default-partial index (sig after backfill so the
632 # UNIQUE constraint only applies once rows are populated). On fresh DBs
633 # _SCHEMA creates it; on migrated DBs this is the first time.
634 await conn.execute(
635     "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
636     "ON policies(account_id) WHERE is_default = 1"
637 )
638
639 # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
640 # produce a content_items row. A mismatch signals fingerprint collisions
641 # where two distinct videos shared (size,dur,h,w) ? the losers stay at
642 # tg:msg: keys and are still present. Log a warning so the operator can
643 # investigate; the data is preserved in *_deprecated tables.
644 async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
645     vc = (await cur.fetchone())[0]
646 async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
647     cic = (await cur.fetchone())[0]
648 if vc != cic:
649     import logging
650
651     _logger = logging.getLogger(__name__)
652     _logger.warning(
653         "migration_003: %d videos rows produced %d content_items rows "
654         "%d gap). Colliding items remain in videos_deprecated.",
655         vc,
656         cic,
657         vc - cic,
658     )

```

```

659
660 # 6. Rename (don't drop) the legacy tables ? kept for one release as a
661 # rollback safety net (ADR 0010 pattern).
662 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
663 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
664 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
665
666
667 @_migration(4)
668 async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
669     """Backfill default destinations and delivery state (ADR 0011, Track C).
670
671     Creates a default destination row for each account that has a
672     ``filter_config_deprecated.delivery_target`` value, then creates a default
673     routing rule (source ? default policy ? destination) for each source.
674     Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
675     table (one ``delivered`` attempt per previously-forwarded video to the
676     account's default destination).
677
678     Idempotent: skips accounts whose default destination already exists.
679     """
680     # Gate: if destinations already have data, this migration has run.
681     async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
682         if (await cur.fetchone())[0] > 0:
683             return
684
685     if not await _table_exists(conn, "filter_config_deprecated"):
686         return
687
688     # 1. Backfill default destinations from filter_config_deprecated.
689     # Parse the JSON delivery_target, defaulting to "me".
690     await conn.execute(
691         """
692         INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
693         is_output, created_at)
694         SELECT user_id, 'telegram',
695             COALESCE(
696                 CASE WHEN json_valid(config_json)
697                     THEN json_extract(config_json, '$.delivery_target') END,
698                 'me'
699             ),
700             CASE
701                 WHEN COALESCE(
702                     CASE WHEN json_valid(config_json)
703                         THEN json_extract(config_json, '$.delivery_target') END,
704                     'me'
705                 ) = 'me'
706                 THEN 'saved'
707                 ELSE 'channel'
708             END,
709             0,
710             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
711         FROM filter_config_deprecated
712         """
713     )
714
715     # 2. Create a default routing rule per source (source ? default policy ? default
716     destination).
717     await conn.execute(
718         """
719         INSERT OR IGNORE INTO routing_rules
720         (account_id, source_id, policy_id, destination_id, enabled, priority,
721         created_at)
722         SELECT s.account_id, s.id, p.id, d.id, 1, 0,
723             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')

```



```

721         FROM sources s
722         JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
723         JOIN destinations d ON d.account_id = s.account_id
724         WHERE d.target = (
725             SELECT COALESCE(
726                 CASE WHEN json_valid(fcd.config_json)
727                     THEN json_extract(fcd.config_json, '$.delivery_target') END,
728                 'me'
729             )
730             FROM filter_config_deprecated fcd
731             WHERE fcd.user_id = s.account_id
732             LIMIT 1
733         )
734         """"
735     )
736
737 # 3. Backfill delivery_attempts from previously-forwarded videos.
738 await conn.execute(
739     """"
740     INSERT OR IGNORE INTO delivery_attempts
741         (content_item_id, destination_id, route_id, status, attempt_count,
742          created_at, delivered_at)
743     SELECT ci.id, d.id, rr.id, 'delivered', 1,
744            vd.seen_at, vd.seen_at
745     FROM videos_deprecated vd
746     JOIN content_items ci
747         ON ci.account_id = vd.user_id
748         AND ci.provider = 'telegram'
749         AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
750     JOIN sources sc
751         ON sc.id = ci.source_id
752         AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
753     JOIN destinations d ON d.account_id = vd.user_id
754     JOIN routing_rules rr
755         ON rr.account_id = vd.user_id
756         AND rr.source_id = sc.id
757         AND rr.destination_id = d.id
758     WHERE vd.forwarded = 1
759     """"
760 )
761
762
763 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
764     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
765     async with conn.execute(
766         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
767     ) as cur:
768         return await cur.fetchone() is not None
769
770
771 # --- helpers -----
772
773
774 def _now() -> datetime:
775     """Timezone-aware UTC 'now', used for policy timestamps."""
776     return datetime.now(UTC)
777
778
779 def _safe_int_cast(value: object, field_name: str) -> int:
780     """Cast a provider_source_id / provider_message_id / document_id to int.
781
782     All existing data is Telegram ? these ids are numeric strings. Non-Telegram
783     providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
784     shapes (which carry Telegram-specific int ids); this function raises a clear
785     ``ValueError`` rather than letting a cryptic ``int()`` crash propagate

```

```

786 (ADR 0011 provider-neutrality guard ? review finding #5).
787 """
788 try:
789     return int(value) # type: ignore[arg-type]
790 except (ValueError, TypeError) as exc:
791     raise ValueError(
792         f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
793         f"types are Telegram-specific; use ContentItem/Source directly for "
794         f"non-Telegram providers (ADR 0011)."
795     ) from exc
796
797
798 def _row_to_account(row: aiosqlite.Row) -> Account:
799     return Account(
800         id=row["id"],
801         telegram_id=row["telegram_id"],
802         provider=row["provider"],
803         display_name=row["display_name"],
804         created_at=row["created_at"], # stored as ISO string; pydantic parses
805     )
806
807
808 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
809     return TelegramSession(
810         account_id=row["account_id"],
811         phone=row["phone"],
812         session_path=row["session_path"],
813         enabled=bool(row["enabled"]),
814         created_at=row["created_at"],
815         last_login_at=row["last_login_at"],
816     )
817
818
819 def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
820     """Project an (account, optional session) pair onto the legacy ``User`` view.
821
822     Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
823     shims so pre-ADR-0010 callers and tests keep working. A bot-only account
824     (no session) maps to a ``User`` with ``enabled=False`` so the legacy
825     "list enabled users" semantics exclude it ? matching the structural rule
826     that bot-only users are not runnable Telethon clients.
827     """
828     if session is None:
829         return User(
830             id=account.telegram_id if account.telegram_id is not None else account.id,
831             phone=None,
832             enabled=False,
833             created_at=account.created_at,
834         )
835     return User(
836         id=account.telegram_id if account.telegram_id is not None else account.id,
837         phone=session.phone,
838         enabled=session.enabled,
839         created_at=session.created_at,
840     )
841
842
843 def _row_to_source(row: aiosqlite.Row) -> Source:
844     return Source(
845         id=row["id"],
846         account_id=row["account_id"],
847         provider=row["provider"],
848         provider_source_id=row["provider_source_id"],
849         title=row["title"],

```

```

850         kind=row["kind"],
851         watched=bool(row["watched"]),
852         seen_first_at=row["seen_first_at"],
853         last_seen_at=row["last_seen_at"],
854     )
855
856
857 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
858     return ContentItem(
859         id=row["id"],
860         account_id=row["account_id"],
861         source_id=row["source_id"],
862         provider=row["provider"],
863         provider_message_id=row["provider_message_id"],
864         content_type=row["content_type"],
865         text=row["text"],
866         link_url=row["link_url"],
867         media_ref=row["media_ref"],
868         document_id=row["document_id"],
869         duration_s=row["duration_s"],
870         width=row["width"],
871         height=row["height"],
872         size_bytes=row["size_bytes"],
873         mime_type=row["mime_type"],
874         dedup_key=row["dedup_key"],
875         matched=bool(row["matched"]),
876         seen_at=row["seen_at"],
877     )
878
879
880 def _row_to_policy(row: aiosqlite.Row) -> Policy:
881     return Policy(
882         id=row["id"],
883         account_id=row["account_id"],
884         name=row["name"],
885         config_json=row["config_json"],
886         is_default=bool(row["is_default"]),
887         created_at=row["created_at"],
888         updated_at=row["updated_at"],
889     )
890
891
892 def _row_to_destination(row: aiosqlite.Row) -> Destination:
893     return Destination(
894         id=row["id"],
895         account_id=row["account_id"],
896         provider=row["provider"],
897         target=row["target"],
898         kind=row["kind"],
899         title=row["title"],
900         is_output=bool(row["is_output"]),
901         created_at=row["created_at"],
902     )
903
904
905 def _row_to_routing_rule(row: aiosqlite.Row) -> RoutingRule:
906     return RoutingRule(
907         id=row["id"],
908         account_id=row["account_id"],
909         source_id=row["source_id"],
910         policy_id=row["policy_id"],
911         destination_id=row["destination_id"],
912         enabled=bool(row["enabled"]),
913         priority=row["priority"],
914         created_at=row["created_at"],

```

```

915         )
916
917
918     def _row_to_delivery_attempt(row: aiosqlite.Row) -> DeliveryAttempt:
919         return DeliveryAttempt(
920             id=row["id"],
921             content_item_id=row["content_item_id"],
922             destination_id=row["destination_id"],
923             route_id=row["route_id"],
924             status=DeliveryStatus(row["status"]),
925             attempt_count=row["attempt_count"],
926             last_error=row["last_error"],
927             created_at=row["created_at"],
928             delivered_at=row["delivered_at"],
929             next_retry_at=row["next_retry_at"],
930         )
931
932
933     # --- account + session repos (ADR 0010) -----
934
935
936     async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
937         """Insert or update an account row; returns the persisted Account."""
938         await conn.execute(
939             """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
940             VALUES (?, ?, ?, ?, ?)
941             ON CONFLICT(id) DO UPDATE SET
942                 telegram_id=excluded.telegram_id,
943                 provider=excluded.provider,
944                 display_name=excluded.display_name""",
945             (
946                 account.id,
947                 account.telegram_id,
948                 account.provider,
949                 account.display_name,
950                 account.created_at.isoformat(),
951             ),
952         )
953         await conn.commit()
954         return await get_account(conn, account.id) # type: ignore[return-value]
955
956
957     async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:
958         conn.row_factory = aiosqlite.Row
959         async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as cur:
960             row = await cur.fetchone()
961             return _row_to_account(row) if row else None
962
963
964     async def get_account_by_telegram_id(
965         conn: aiosqlite.Connection, telegram_id: int
966     ) -> Account | None:
967         """Resolve a Bot API ``from.id`` (or userbot account id) to an Account."""
968         conn.row_factory = aiosqlite.Row
969         async with conn.execute("SELECT * FROM accounts WHERE telegram_id = ?",
970 (telegram_id,)) as cur:
971             row = await cur.fetchone()
972             return _row_to_account(row) if row else None
973
974
975     async def upsert_telegram_session(
976         conn: aiosqlite.Connection, session: TelegramSession
977     ) -> TelegramSession:
978         """Insert or update a telegram_sessions row; returns the persisted session."""

```

```

978     await conn.execute(
979         """INSERT INTO telegram_sessions
980             (account_id, phone, session_path, enabled, created_at, last_login_at)
981             VALUES (?, ?, ?, ?, ?, ?)
982             ON CONFLICT(account_id) DO UPDATE SET
983                 phone=excluded.phone,
984                 session_path=excluded.session_path,
985                 enabled=excluded.enabled,
986                 last_login_at=excluded.last_login_at""",
987         (
988             session.account_id,
989             session.phone,
990             session.session_path,
991             int(session.enabled),
992             session.created_at.isoformat(),
993             session.last_login_at.isoformat() if session.last_login_at else None,
994         ),
995     )
996     await conn.commit()
997     conn.row_factory = aiosqlite.Row
998     async with conn.execute(
999         "SELECT * FROM telegram_sessions WHERE account_id = ?", (session.account_id,)
1000     ) as cur:
1001         row = await cur.fetchone()
1002         # The row must exist ? we just upserted it by PK (account_id). Assert the
1003         # invariant explicitly so mypy can narrow Row | None ? Row without a
1004         # type-ignore comment masking a real future regression.
1005         assert row is not None
1006         return _row_to_telegram_session(row)
1007
1008
1009     async def get_telegram_session(
1010         conn: aiosqlite.Connection, account_id: int
1011     ) -> TelegramSession | None:
1012         conn.row_factory = aiosqlite.Row
1013         async with conn.execute(
1014             "SELECT * FROM telegram_sessions WHERE account_id = ?", (account_id,)
1015         ) as cur:
1016             row = await cur.fetchone()
1017             return _row_to_telegram_session(row) if row else None
1018
1019
1020     async def list_runnable_sessions(
1021         conn: aiosqlite.Connection,
1022     ) -> list[tuple[Account, TelegramSession]]:
1023         """Return (account, session) pairs for sessions the manager should start.
1024
1025         This is the ADR 0010 structural rule: ``manager.run`` iterates this list,
1026         not ``accounts``. A bot-only user has no ``telegram_sessions`` row and is
1027         therefore inherently invisible to the Telethon supervisor ? the "don't
1028         start a client for bot-only users" invariant is enforced by query shape.
1029         """
1030         conn.row_factory = aiosqlite.Row
1031         async with conn.execute(
1032             """SELECT a.*, s.phone AS s_phone, s.session_path, s.enabled AS s_enabled,
1033                 s.created_at AS s_created_at, s.last_login_at
1034             FROM accounts a
1035             JOIN telegram_sessions s ON s.account_id = a.id
1036             WHERE s.enabled = 1"""
1037         ) as cur:
1038             rows = await cur.fetchall()
1039             return [
1040                 (
1041                     _row_to_account(r),
1042                     TelegramSession(

```

```

1043         account_id=r["id"],
1044         phone=r["s_phone"],
1045         session_path=r["session_path"],
1046         enabled=bool(r["s_enabled"]),
1047         created_at=r["s_created_at"],
1048         last_login_at=r["last_login_at"],
1049     ),
1050 )
1051 for r in rows
1052 ]
1053
1054
1055 # --- legacy User shims (backwards-compatible view over accounts + sessions) --
1056
1057
1058 async def upsert_user(conn: aioredis.Connection, user: User) -> User:
1059     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
1060
1061     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
1062     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
1063     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
1064     the manager can run them. ``User.enabled`` maps to ``session.enabled``.
1065     """
1066     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
1067     await upsert_account(conn, account)
1068     await upsert_telegram_session(
1069         conn,
1070         TelegramSession(
1071             account_id=user.id,
1072             phone=user.phone,
1073             session_path=f"user_{user.id}.session",
1074             enabled=user.enabled,
1075             created_at=user.created_at,
1076         ),
1077     )
1078     return await get_user(conn, user.id) # type: ignore[return-value]
1079
1080
1081 async def get_user(conn: aioredis.Connection, user_id: int) -> User | None:
1082     """Deprecated shim: project the (account, session) pair back to a ``User``."""
1083     account = await get_account(conn, user_id)
1084     if account is None:
1085         return None
1086     session = await get_telegram_session(conn, user_id)
1087     return _account_session_to_user(account, session)
1088
1089
1090 async def list_enabled_users(conn: aioredis.Connection) -> list[User]:
1091     """Deprecated shim: list users that have an enabled MTPROTO session.
1092
1093     Equivalent to projecting :func:`list_runnable_sessions` onto the legacy
1094     ``User`` shape. Bot-only accounts (no session) are excluded ? matching the
1095     pre-ADR-0010 "list enabled users" semantics where every row was runnable.
1096     """
1097     pairs = await list_runnable_sessions(conn)
1098     return [_account_session_to_user(a, s) for a, s in pairs]
1099
1100
1101 # --- Source repos (ADR 0011) -----
1102
1103
1104 async def upsert_source(conn: aioredis.Connection, source: Source) -> Source:
1105     """Insert a source row, or refresh title/last_seen_at on conflict.
1106
1107     The conflict key is ``(account_id, provider, provider_source_id)`` per the

```

```

1108     ADR 0011 schema. Returns the persisted row.
1109
1110     ``watched`` is **user-controlled state** (toggled only via
1111     :func:`set_source_watched` / the ``/watch`` and ``/unwatch`` commands) and is
1112     therefore **never overwritten on conflict** ? the ingestion path
1113     (``upsert_channel`` / ``insert_video``) re-touches a source on every message,
1114     so clobbering ``watched`` here would silently un-watch a channel the user
1115     opted into after its first ingested video (ADR 0009). ``watched`` is only set
1116     on the initial INSERT of a new row. ``title`` is refreshed but never nulled:
1117     the ingestion shims pass ``title=None``, so ``COALESCE`` keeps the existing
1118     title rather than erasing the channel name.
1119     """
1120     await conn.execute(
1121         """INSERT INTO sources
1122             (account_id, provider, provider_source_id, title, kind, watched,
1123              seen_first_at, last_seen_at)
1124             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1125             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1126                 title=COALESCE(excluded.title, sources.title),
1127                 last_seen_at=excluded.last_seen_at""",
1128         (
1129             source.account_id,
1130             source.provider,
1131             source.provider_source_id,
1132             source.title,
1133             source.kind,
1134             int(source.watched),
1135             source.seen_first_at.isoformat(),
1136             (source.last_seen_at.isoformat() if source.last_seen_at else None),
1137         ),
1138     )
1139     await conn.commit()
1140     conn.row_factory = aiosqlite.Row
1141     async with conn.execute(
1142         """SELECT * FROM sources
1143             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1144         (source.account_id, source.provider, source.provider_source_id),
1145     ) as cur:
1146         row = await cur.fetchone()
1147         assert row is not None # just upserted by the conflict key
1148         return _row_to_source(row)
1149
1150
1151     async def get_source_by_provider_id(
1152         conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
1153     ) -> Source | None:
1154         """Resolve a source by its provider-native id."""
1155         conn.row_factory = aiosqlite.Row
1156         async with conn.execute(
1157             """SELECT * FROM sources
1158                 WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1159             (account_id, provider, provider_source_id),
1160         ) as cur:
1161             row = await cur.fetchone()
1162             return _row_to_source(row) if row else None
1163
1164
1165     # --- ContentItem repos (ADR 0011) -----
1166
1167
1168     async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
1169         """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1170
1171         The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two

```

```

1172 unique constraints guard against duplicates:
1173 - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1174 - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1175 """
1176 cur = await conn.execute(
1177     """INSERT INTO content_items
1178         (account_id, source_id, provider, provider_message_id, content_type,
1179          text, link_url, media_ref, document_id, duration_s, width, height,
1180          size_bytes, mime_type, dedup_key, matched, seen_at)
1181         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
1182     (
1183         item.account_id,
1184         item.source_id,
1185         item.provider,
1186         item.provider_message_id,
1187         item.content_type,
1188         item.text,
1189         item.link_url,
1190         item.media_ref,
1191         item.document_id,
1192         item.duration_s,
1193         item.width,
1194         item.height,
1195         item.size_bytes,
1196         item.mime_type,
1197         item.dedup_key,
1198         int(item.matched),
1199         item.seen_at.isoformat(),
1200     ),
1201 )
1202 await conn.commit()
1203 return item.model_copy(update={"id": cur.lastrowid})
1204
1205
1206 async def get_content_item_by_message(
1207     conn: aiosqlite.Connection,
1208     account_id: int,
1209     source_id: int,
1210     provider_message_id: str,
1211 ) -> ContentItem | None:
1212     """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
1213     conn.row_factory = aiosqlite.Row
1214     async with conn.execute(
1215         """SELECT * FROM content_items
1216             WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
1217         (account_id, source_id, provider_message_id),
1218     ) as cur:
1219         row = await cur.fetchone()
1220         return _row_to_content_item(row) if row else None
1221
1222
1223 async def get_content_item_by_dedup_key(
1224     conn: aiosqlite.Connection, account_id: int, dedup_key: str
1225 ) -> ContentItem | None:
1226     """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1227
1228     Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1229     a single query: the ``dedup_key`` already encodes whichever tier was
1230     applicable at ingestion time.
1231     """
1232     conn.row_factory = aiosqlite.Row
1233     async with conn.execute(
1234         "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1235         (account_id, dedup_key),
1236     ) as cur:

```



```

1237         row = await cur.fetchone()
1238         return _row_to_content_item(row) if row else None
1239
1240
1241 # --- Policy repos (ADR 0011, Option 2A) -----
1242
1243
1244 async def upsert_policy(conn: aiomysqlite.Connection, policy: Policy) -> Policy:
1245     """Insert or update a policy row by `(account_id, name)`".
1246
1247     Exactly one policy per account should carry `is_default=True`; callers
1248     are responsible for flipping the previous default before flipping a new
1249     one (enforced by a partial unique index landed by future work, not in the
1250     schema ? SQLite `CHECK` can't express "at most one per group").
1251     """
1252     await conn.execute(
1253         """INSERT INTO policies (account_id, name, config_json, is_default,
1254                                created_at, updated_at)
1255                                VALUES (?, ?, ?, ?, ?, ?)
1256                                ON CONFLICT(account_id, name) DO UPDATE SET
1257                                    config_json=excluded.config_json,
1258                                    is_default=excluded.is_default,
1259                                    updated_at=excluded.updated_at""",
1260         (
1261             policy.account_id,
1262             policy.name,
1263             policy.config_json,
1264             int(policy.is_default),
1265             policy.created_at.isoformat(),
1266             policy.updated_at.isoformat(),
1267         ),
1268     )
1269     await conn.commit()
1270     conn.row_factory = aiomysqlite.Row
1271     async with conn.execute(
1272         "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1273         (policy.account_id, policy.name),
1274     ) as cur:
1275         row = await cur.fetchone()
1276         assert row is not None
1277         return _row_to_policy(row)
1278
1279
1280 async def get_default_policy(conn: aiomysqlite.Connection, account_id: int) -> Policy |
None:
1281     """Return the account's default policy (`is_default=1`)."
1282     conn.row_factory = aiomysqlite.Row
1283     async with conn.execute(
1284         "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1285         (account_id,),
1286     ) as cur:
1287         row = await cur.fetchone()
1288         return _row_to_policy(row) if row else None
1289
1290
1291 # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1292
1293
1294 async def upsert_destination(conn: aiomysqlite.Connection, dest: Destination) ->
Destination:
1295     """Insert or update a destination row by (account, provider, target)."
1296     await conn.execute(
1297         """INSERT INTO destinations
1298            (account_id, provider, target, kind, title, is_output, created_at)
1299            VALUES (?, ?, ?, ?, ?, ?, ?)

```

```

1300         ON CONFLICT(account_id, provider, target) DO UPDATE SET
1301             kind=excluded.kind,
1302             title=excluded.title,
1303             is_output=excluded.is_output"",
1304         (
1305             dest.account_id,
1306             dest.provider,
1307             dest.target,
1308             dest.kind,
1309             dest.title,
1310             int(dest.is_output),
1311             dest.created_at.isoformat(),
1312         ),
1313     )
1314     await conn.commit()
1315     conn.row_factory = aisqlite.Row
1316     async with conn.execute(
1317         """SELECT * FROM destinations
1318             WHERE account_id = ? AND provider = ? AND target = ?""",
1319         (dest.account_id, dest.provider, dest.target),
1320     ) as cur:
1321         row = await cur.fetchone()
1322         assert row is not None
1323         return _row_to_destination(row)
1324
1325
1326     async def get_destination(
1327         conn: aisqlite.Connection, account_id: int, provider: str, target: str
1328     ) -> Destination | None:
1329         """Resolve a destination by its (account, provider, target) natural key."""
1330         conn.row_factory = aisqlite.Row
1331         async with conn.execute(
1332             "SELECT * FROM destinations WHERE account_id = ? AND provider = ? AND target = ?"
1333             ,
1334             (account_id, provider, target),
1335         ) as cur:
1336             row = await cur.fetchone()
1337             return _row_to_destination(row) if row else None
1338
1339     async def record_delivery(
1340         conn: aisqlite.Connection,
1341         *,
1342         content_item_id: int,
1343         destination_id: int,
1344         route_id: int | None = None,
1345         success: bool,
1346         error: str | None = None,
1347     ) -> None:
1348         """Record a delivery outcome for one (content_item, destination) pair.
1349
1350         This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
1351         sets status=delivered; on failure, increments attempt_count and records the
1352         error. Idempotent: repeated calls update the same row.
1353         """
1354         status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
1355         await conn.execute(
1356             """INSERT INTO delivery_attempts
1357                 (content_item_id, destination_id, route_id, status, attempt_count,
1358                  last_error, created_at, delivered_at)
1359                 VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
1360                          CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
1361                 ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
1362                     status=excluded.status,
1363                     attempt_count=delivery_attempts.attempt_count + 1,

```

```

1364         last_error=excluded.last_error,
1365         delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
1366                 ELSE delivery_attempts.delivered_at END""",
1367         (
1368             content_item_id,
1369             destination_id,
1370             route_id,
1371             status.value,
1372             error,
1373             success,
1374             success,
1375         ),
1376     )
1377     await conn.commit()
1378
1379
1380     async def get_delivery_attempt(
1381         conn: aiomysqlite.Connection, content_item_id: int, destination_id: int
1382     ) -> DeliveryAttempt | None:
1383         """Read a delivery attempt row by its natural key (idempotency boundary)."""
1384         conn.row_factory = aiomysqlite.Row
1385         async with conn.execute(
1386             "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id =
1387             ?",
1388             (content_item_id, destination_id),
1389         ) as cur:
1390             row = await cur.fetchone()

```

6. assistant (2026-07-06T19:16:09.411Z)

I have the key files. Let me verify the specific claims with Python and read the save/load shims plus the telethon handler.

7. user (2026-07-06T19:16:11.108Z)

```

1698:async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1711:async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:

```

8. user (2026-07-06T19:16:16.325Z)

```

1690         forwarded=False,
1691         seen_at=item.seen_at,
1692     )
1693
1694
1695     # --- filter config shims (backwards-compatible view over policies) -----
1696
1697
1698     async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1699         """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1700
1701         ADR 0011 (Option 2A): ``filter_config`` is renamed to
1702         ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1703         instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1704         """
1705         policy = await get_default_policy(conn, user_id)
1706         if policy is None:
1707             return FilterConfig()
1708         return FilterConfig.model_validate_json(policy.config_json)
1709
1710
1711     async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:

```

```

1712     """Deprecated shim: upsert the account's default :class:`Policy`."""
1713     existing = await get_default_policy(conn, user_id)
1714     await upsert_policy(
1715         conn,
1716         Policy(
1717             id=existing.id if existing is not None else None,
1718             account_id=user_id,
1719             name="default",
1720             config_json=cfg.model_dump_json(),
1721             is_default=True,
1722             created_at=(existing.created_at if existing is not None else _now()),
1723             updated_at=_now(),
1724         ),
1725     )
1726
1727
1728 # --- bulk helpers (used by the manager on startup) -----
1729
1730
1731 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1732     conn: aiosqlite.Connection, user_id: int, limit: int = 100
1733 ) -> list[VideoRecord]:
1734     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1735     conn.row_factory = aiosqlite.Row
1736     async with conn.execute(
1737         """SELECT ci.*, s.provider_source_id AS src_pid
1738            FROM content_items ci
1739           JOIN sources s ON s.id = ci.source_id
1740          WHERE ci.account_id = ?
1741                 ORDER BY ci.id DESC LIMIT ?""",
1742         (user_id, limit),
1743     ) as cur:
1744         rows = await cur.fetchall()
1745     return [
1746         VideoRecord(
1747             id=r["id"],
1748             user_id=user_id,
1749             channel_id=safe_int_cast(r["src_pid"], "provider_source_id"),
1750             message_id=safe_int_cast(r["provider_message_id"], "provider_message_id"),
1751             document_id=(
1752                 _safe_int_cast(r["document_id"], "document_id")
1753                 if r["document_id"] is not None
1754                 else None
1755             ),
1756             duration_s=r["duration_s"],
1757             width=r["width"],
1758             height=r["height"],
1759             size_bytes=r["size_bytes"],
1760             mime_type=r["mime_type"],
1761             matched=bool(r["matched"]),
1762             forwarded=False,
1763             seen_at=r["seen_at"],
1764         )
1765         for r in rows
1766     ]
1767
1768
1769 __all__ = [
1770     "init_db",
1771     "compute_dedup_key",
1772     "_safe_int_cast",
1773     # Account + session repos (ADR 0010)
1774     "upsert_account",
1775     "get_account",
1776     "get_account_by_telegram_id",

```

```

1777     "upsert_telegram_session",
1778     "get_telegram_session",
1779     "list_runnable_sessions",
1780     # Provider-neutral domain repos (ADR 0011)
1781     "upsert_source",
1782     "get_source_by_provider_id",
1783     "get_source_by_id",
1784     "set_source_watched",
1785     "list_account_sources",
1786     "should_ingest_source",
1787     "insert_content_item",
1788     "get_content_item_by_message",
1789     "get_content_item_by_dedup_key",
1790     "upsert_policy",
1791     "get_default_policy",
1792     # Destination / Routing / Delivery repos (ADR 0011, Track C)
1793     "upsert_destination",
1794     "get_destination",
1795     "record_delivery",
1796     "get_delivery_attempt",
1797     # Legacy User shims (backwards-compatible)
1798     "upsert_user",
1799     "get_user",
1800     "list_enabled_users",
1801     # Legacy Channel / Video / config shims (ADR 0011)
1802     "upsert_channel",
1803     "get_channel",
1804     "get_video",
1805     "get_video_by_document_id",
1806     "get_video_by_content_fingerprint",
1807     "insert_video",
1808     "mark_forwarded",
1809     "load_filter_config",
1810     "save_filter_config",
1811     "list_videos_for_user",
1812 ]
1813
1814
1815 def __dir__() -> Iterable[str]:
1816     return __all__

```

9. assistant (2026-07-06T19:16:18.723Z)

This confirms the SEED. Now let me verify the round-trip data-loss with Python and check the telethon handler for how `text` (caption) is populated into ContentItem.

10. user (2026-07-06T19:16:24.270Z)

```

parsed OK; has topics attr: False
topics survived round-trip: False
allowlist survived: False
deep_inspect survived: False
keys after round-trip: ['delivery_mode', 'delivery_target', 'exclude_gif', 'exclude_nosound',
'exclude_round', 'max_duration_s', 'max_size_mb', 'min_duration_s', 'target_height']

```

11. assistant (2026-07-06T19:16:27.308Z)

SEED confirmed: extra keys are silently dropped on both read and re-serialize. Now let me check `topics` object-vs-array, and the telethon handler for whether `content\_item.text` (caption) is populated (Tier 2/3 depend on it).

12. user (2026-07-06T19:16:31.605Z)

example INVALID JSON: Expecting ',' delimiter: line 6 column 20 (char 144)

13. user (2026-07-06T19:16:32.090Z)

```
313:             link_text=link_text,
```

14. assistant (2026-07-06T19:16:35.887Z)

The `` comment makes the example invalid JSON ? confirmed. Now let me see how the ContentItem is built in the handler to check whether `text` (caption) is populated.

15. user (2026-07-06T19:16:37.753Z)

```
6-         -> extract_video_meta(message)    # pure, unit-tested
7-         -> dedup via db.get_video()
8-         -> FilterEngine.evaluate(meta)
9-         -> db.insert_video(...) + delivery.forward_to_target(..., target=cfg.delivery_target)
10-
11-The metadata extraction is deliberately a pure function taking a Telethon
--
34-from tg_video_filter import db
35-from tg_video_filter.commands import ENGINE_KEY
36-from tg_video_filter.delivery import build_link_text, deliver
37-from tg_video_filter.filters import FilterEngine, VideoMeta
38-from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User, VideoRecord
39-
--
148-     user: User,
149-     conn: aiosqlite.Connection,
150-     engine_holder: dict[str, FilterEngine],
151-     client: TelegramClient,
152-     delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
153-     delivery_target: str = "me",
154-) -> _NewMessageCallback:
155-     """Return an async callback for ``events.NewMessage``.
--
162-         changes via ``/set`` take effect immediately),
163-         4. inserts a VideoRecord row,
164-         5. delivers a match to ``delivery_target`` (see ADR 0005).
165-     """
166-
--
187-         # unwatched and lazily create a watched=0 row for /channels
188-         # discoverability (ADR 0009 lines 37-44, 130-131).
189-         if not await db.should_ingest_source(conn, user.id, channel_id):
190-             return # skip unwatched / unknown source
191-
--
255-         )
256-
257-         result = engine_holder[ENGINE_KEY].evaluate(meta)
258-         record = VideoRecord(
259-             user_id=user.id,
--
271-             inserted = await db.insert_video(conn, record)
272-         except sqlite3.IntegrityError:
273-             # Lost a dedup race: a concurrent delivery of the same file
274-             # (same document_id or same message instance) won the insert.
275-             # The three-tier check above already classified this as new, so
--
294-
295-         if result.matched:
296-             # Build link text for LINK/BOTH modes (None for FORWARD).
297-             link_text = None
298-             if delivery_mode != DeliveryMode.FORWARD:
299-                 channel_title = _channel_title(event)
300-                 link_text = build_link_text(
```

```

--
305-         channel_title=channel_title,
306-     )
307-     await deliver(
308-         client,
309-         message,
310-         conn,
311-         inserted.id,
312-         delivery_mode,
313-         link_text=link_text,
314-         target=delivery_target,
315-     )
316-

```

16. assistant (2026-07-06T19:16:41.736Z)

The live handler uses the `**legacy `VideoRecord`/`insert_video` path**` (not ``ContentItem`/`insert_content_item``), and does not populate any caption/text field. Let me confirm ``VideoRecord`` has no text field (already seen: it doesn't) and check the F1 claim about wiring into the handler. Let me look at the `extract_video_meta` and whether caption text is even captured.

17. user (2026-07-06T19:16:42.796Z)

```

255         )
256
257     result = engine_holder[ENGINE_KEY].evaluate(meta)
258     record = VideoRecord(
259         user_id=user.id,
260         channel_id=channel_id,
261         message_id=message_id,
262         document_id=document_id,
263         duration_s=meta.duration_s,
264         width=meta.width,
265         height=meta.height,
266         size_bytes=meta.size_bytes,
267         mime_type=meta.mime_type,
268         matched=result.matched,
269     )
270     try:
271         inserted = await db.insert_video(conn, record)
272     except sqlite3.IntegrityError:
273         # Lost a dedup race: a concurrent delivery of the same file
274         # (same document_id or same message instance) won the insert.
275         # The three-tier check above already classified this as new, so
276         # the collision means another task is processing (or has
277         # processed) it ? log and skip.
278         log.info(
279             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
280             user.id,
281             channel_id,
282             message_id,
283             document_id,
284         )
285         return
286     log.info(
287         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
288         user.id,
289         channel_id,
290         message_id,
291         result.matched,
292         result.reasons,
293     )
294

```

18. assistant (2026-07-06T19:16:48.885Z)

Confirmed. The live handler builds a `VideoRecord` (no `text`/caption), uses `insert\_video`, and never populates `ContentItem.text`. Now let me quickly verify the ADR's claim about ADR 0009's `topics.allowlist` reference and check the FilterEngine.evaluate signature (ADR says "extends today's FilterEngine" / F1 wires PolicyContext with ContentItem+Source+Policy ? but handler has none of these on hand). Let me check the ADR 0003 and 0011 references briefly and the FilterEngine.evaluate input.

19. user (2026-07-06T19:16:51.954Z)

```
evaluate sig: (self, meta: 'VideoMeta') -> 'FilterResult'
VideoMeta fields: dict_keys(['duration_s', 'width', 'height', 'size_bytes', 'mime_type',
'is_gif', 'is_round', 'nosound'])
```

20. assistant (2026-07-06T19:16:56.860Z)

Now I have enough. Let me verify one more claim ? the ADR says dedup runs before the pipeline and Tier 3 "caption text scanning" / Tier 2 "message caption keyword match" rely on caption text, but `ContentItem.text` is documented as "nullable in Phase 1" and never populated by the live path. That's a real gap for F2/F4. I have all findings confirmed. Let me also quickly confirm the ADR's "Depends on ADR 0011" and the 0009 allowlist reference wording by checking ADR 0009 exists.

21. user (2026-07-06T19:16:59.619Z)

ADR 0009: Selective channel scanning and multiplexed routing

```
- **Status**: Accepted
- **Date**: 2026-07-04
- **Decision owner**: project owner
---0003---
```

ADR 0003: Metadata-first probing (no download in the hot path)

```
- **Status**: Accepted
- **Date**: 2026-07-03
```

22. assistant (2026-07-06T19:17:03.608Z)

Both referenced ADRs exist. I have all findings confirmed. Let me compile the structured output.

23. user (2026-07-06T19:17:45.506Z)

Structured output provided successfully